

进程通信，理论课作业

同济大学计算机系操作系统作业 2024 拔尖班、国豪班

2024/12/13

姓名 毛星博

学号 2351520

一、选择、填空

(一) 临界区、临界资源

1. 临界区是指 D。
A. 并发进程中用于实现进程互斥的程序段
B. 并发进程中用于实现进程同步的程序段
C. 并发进程中用户实现进程通信的程序段
D. 并发进程中访问共享变量的程序段
2. 相关临界区是指 D。
A. 一个独占资源
B. 并发进程中与共享变量有关的程序段
C. 一个共享资源
D. 并发进程中涉及相同变量的那些程序段
3. 管理若干进程共享某一资源的相关临界区应满足三个要求，其中不考虑 A。
A. 一个进程可以抢占已分配给另一进程的资源
B. 任何进程不应该无限地逗留在它的临界区中
C. 一次最多让一个进程在临界区执行
D. 不能强迫一个进程无限地等待进入它的临界区
注释：参考互斥机制应该遵循的原则。Part5 进程通信 1，第 18 页。
4. 若一个进程已进入临界区，其它欲进入临界区的进程必须 等待。

(二) 信号量 和 P、V 操作

1. D 是一种只能进行 P 操作和 V 操作的特殊变量。
A. 调度
B. 进程
C. 同步
D. 信号量
2. P、V 操作是 A。
A. 两条低级进程通信原语
B. 两组不同的机器指令
C. 两条系统调用命令
D. 两条高级进程通信原语
3. 若 P、V 操作的信号量 S 初值为 2，当前值为 -1，则表示有 B 等待进程。
A. 0 个
B. 1 个
C. 2 个
D. 3 个
4. 用 P、V 操作管理互斥资源时，信号量的初值应定义为 C。
A. -1
B. 0
C. 1
D. 任意值

5、用 V 操作唤醒一个等待进程时，被唤醒进程的状态变为 B 。

- A.等待 B. 就绪 C. 运行 D. 完成

6、对于整型信号量，在执行一次 P 操作时，信号量的值应C。

- A. 不变 B. 加 1 C. 减 1 D. 减指定数值

7、在执行 V 操作时，当信号量的值D时，应释放一个等待该信号量的进程。

- A. >0 B. <0 C. ≥ 0 D. ≤ 0

8、P、V 操作必须在屏蔽中断下执行，这种不可被中断的过程称为 B。

- A. 初始化程序 B. 原语（原子操作） C. 子程序 D. 控制模块

9、信号量的物理意义是当信号量值大于零时表示 ①可用资源的数目；当信号量值小于零时，其绝对值为 ②等待使用资源的进程数目。

10、用 P、V 操作管理相关临界区时，必须成对出现，任何一个进程在进入临界区之前应调用 ①P 操作，退出临界区时应调用 ②V 操作。

11、有 m 个进程互斥共享同一临界资源，若使用信号量机制实现对临界资源的互斥访问，则信号量值的变化范围是 $[-(m-1), 1]$ 。

12、操作系统中，对信号量 S 的 P 原语操作定义中，使进程进入相应等待队列等待的条件是 $S.value \leq 0$ 。

13、若信号量的初值为n，用PV操作能限制一次_____进程进入临界区操作。

14. 用 PV 操作实现进程同步时，调用 ①P 操作 测试消息是否到达，调用 ②V 操作 发送消息。

15. 信号量 S 保护共享资源 R。S 的初值是 2，当前值为-1。请问资源 R 有 2 个？当前，1 个进程正在等待使用 R 资源。

(三) 进程间的同步和互斥

1、进程间的同步是指进程间在逻辑上的相互 **B** 关系。

- A. 联接 B. 制约 C. 继续 D. 调用

2、进程间的互斥与同步分别表示了各进程间的 **A**。

- A. 竞争与协作 B. 相互独立与相互制约
C. 不同状态 D. 动态性与并发性

3、实现进程互斥时，用 **C** 对应，对同一个信号量调用 P、V 操作实现互斥。

- A. 一个信号量与一个临界区 B. 一个信号量与一个相关临界区
C. 一个信号量与一组相关临界区 D. 一个信号量与一个消息

4、实现进程同步时，每一个消息与一个信号量对应，进程____D____可把不同的消息发送出去。

- A. 在同一信号量上调用 P 操作 B. 在不同信号量上调用 P 操作
C. 在同一信号量上调用 V 操作 D. 在不同信号量上调用 V 操作

5、进程的互斥是指当有若干进程都要使用某一共享资源时，任何时刻最多只允许一个进程去使用。

6、进程的同步是指并发进程之间存在一种制约关系，一个进程的执行依赖另一个进程的消息（或，一个任务执行完毕，另一个任务才能执行）。

7、进程的互斥实质上是进程同步的一种特殊情况。

8、两个进程访问同一个共享变量，怎样实现把共享变量放在文件或共享内存里
如果未加控制，会出现什么情况？共享变量的值可能出错
保证变量的值不出错，应该怎样做互斥访问，也就是加锁
为此，计算机系统为应用系统提供了一些工具。最常用的是信号量、自旋锁
在编写应用系统的时候，如果没做好互斥，会出现什么情况共享数据结构的价值会出错
同步没做好呢？系统时序不对，无法维护前驱、后继关系，可能发生死锁或进程饿死现象。

（四）与时间有关的错误（或称为竞争条件 race condition）

1. 只要涉及相同变量的若干进程的相关临界区执行时段没有交集，就不会造成与时间有关的错误。

[注释] 时间有关的错误是指多个进程同时对同一个共享变量进行访问时，变量最终的值取决于进程执行的先后次序，有时正确、有时错误。参见下面的例子：

例：假设某游乐场有一个入口和一个出口，管理人员需要随时了解当前游乐场中有多少游客，进程IN和进程OUT分别承担在入口处和出口处统计游客数的任务。当前游客数存放在共享计数器变量count中。

进程IN: count++; // 汇编之后的指令: 1: mov eax, count; 2: inc eax; 3: mov count, eax
进程OUT: count--; // 汇编之后的指令: 4: mov eax, count; 5: dec eax; 6: mov count, eax

T时刻，count的值是1000。有一个游客进入游乐场，同时，一名游客离开。
正确的系统状态应该是这6条指令执行完毕之后，count == 1000。

推进顺序1: 1 4 5 6 2 3。count的值，最终会是1001，减法操作的结果丢了。
推进顺序2: 4 5 1 2 3 6。count的值，最终会是999，加法操作的结果丢了。
推进顺序3: count++完整完成，之后再执行count--操作。或者，count--操作先完成，再执行count++操作。

这种变量最终的值和进程推进速度有关的错误叫做竞争条件(race condition)，又叫与时间有关的错误

二、简答题

- 1、简述临界区和相关临界区的概念。管理相关临界区有些什么要求？

临界区是指并发进程中访问共享变量或共享资源的程序段。

相关临界区是指多个并发进程中涉及同一个共享变量或同一个共享资源的那些程序段。

管理相关临界区通常应满足以下三个要求：

- **互斥进入**：一次最多只允许一个进程在相关临界区中执行。
- **有限停留**：任何进入临界区的进程，都必须在有限时间内退出临界区，不能无限期占用资源。

- 有限等待：不能让某个进程无限等待进入临界区，也就是要避免饥饿现象。简单说，就是：一次只能一个进程进，用完要及时出来，别人不能一直等不到。

2、为什么并发进程执行时可能会产生与时间有关的错误？如何避免？

并发进程执行时，如果多个进程同时访问同一个共享变量或共享数据结构，而且没有采取互斥控制，那么程序执行结果就可能依赖于进程执行的先后顺序，从而产生与时间有关的错误，也称为竞争条件。

例如，一个进程正在修改共享变量时，可能因为中断而放弃 CPU，另一个进程又去访问或修改同一个变量。这样共享变量的最终结果就可能出错。

避免方法是：

对共享数据结构的访问进行序列化，也就是保证同一时刻最多只有一个进程访问该共享数据。具体做法可以是使用信号量、锁、PV 操作等机制实现互斥访问。

3、请给出 PV 操作的定义。假设 PV 操作用信号量 s 管理某个共享资源，请问当 $s.value > 0$ ， $s.value = 0$ 和 $s.value < 0$ 时，它们的物理意义是什么？

P 操作是同步获得共享资源的访问权限。

V 操作是共享资源使用完毕，释放访问权限。

```
procedure P(s)
{
    s.value = s.value - 1;
    if (s.value < 0)
        sleep(s);
}

procedure V(s)
{
    s.value = s.value + 1;
    if (s.value <= 0)
        wakeup(s);
}
```

当 $s.value > 0$ 时，表示资源还有剩余， $s.value$ 的值就是当前可用资源的数量。

当 $s.value = 0$ 时，表示资源刚好全部被占用，没有空闲资源，但也没有进程在等待。

当 $s.value < 0$ 时，表示资源已经用完，并且有进程正在等待该资源。此时 $|s.value|$ 表示等待使用该资源的进程数。

4、用 PV 操作实现进程间同步与互斥应注意些什么？

1.要为共享资源设置合适的信号量。

互斥问题中，通常一个共享资源设置一个互斥信号量；同步问题中，通常每一种前驱后继关系或每一种消息设置一个同步信号量。

2.信号量初值要设置正确。

互斥信号量的初值一般设为 1，表示资源一开始可用；同步信号量的初值一般设为 0，表示某个消息或条件一开始尚未发生。

3.PV 操作必须成对出现。

互斥时，同一个进程通常在进入临界区前执行 P 操作，在退出临界区后执行 V 操作。同步时，P 操作和 V 操作往往出现在不同进程中：等待者执行 P 操作，通知者执行 V 操作。

4.PV 操作的位置必须正确。

P 操作放错位置可能导致进程过早执行或死锁；V 操作放错位置可能导致同步关系被破坏。

5.同步和互斥同时存在时，要注意嵌套顺序。

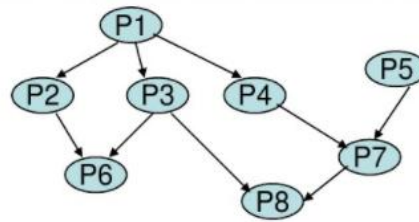
一般来说，同步的 P、V 操作放在外层，互斥的 P、V 操作放在内层。也就是说，先判断是否满足同步条件，再进入互斥访问共享资源的临界区。参考答案中也特别强调了这一点。

三、经典进程通信问题

1、认真看 PPT，掌握 PPT 上介绍过的所有进程同步问题。伪代码填空。

2、用 PV 操作实现任务之间的前驱、后继关系。即，所有前驱任务完成之后，后继任务才能开始执行。

2.1 有 8 个任务，前驱后继关系如下图。P1 任务完成后，可以执行 P2、P3 和 P4 任务；P2 任务和 P3 任务全部执行完毕后，才能够执行 P6 任务。请用 PV 操作控制任务间的前驱后继关系。



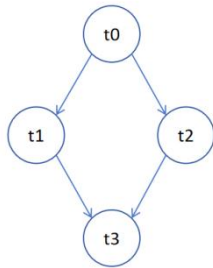
```

1 semaphore s12 = 0;
2 semaphore s13 = 0;
3 semaphore s14 = 0;
4 semaphore s26 = 0;
5 semaphore s36 = 0;
6 semaphore s47 = 0;
7 semaphore s57 = 0;
8 semaphore s38 = 0;
9 semaphore s78 = 0;

```

<pre> P1() { task P1; V(s12); V(s13); V(s14); } </pre>	<pre> P2() { P(s12); task P2; V(s26); } </pre>	<pre> P3() { P(s13); task P3; V(s36); V(s38); } </pre>	<pre> P4() { P(s14); task P4; V(s47); } </pre>
<pre> P5() { task P5; V(s57); } </pre>	<pre> P6() { P(s26); P(s36); task P6; } </pre>	<pre> P7() { P(s47); P(s57); task P7; V(s78); } </pre>	<pre> P8() { P(s38); P(s78); task P8; } </pre>

2.2 某分布式程序由 4 个子任务组成，它们分别为：t0、t1、t2 和 t3。任务之间有严格的前驱后继关系：t1 任务在 t0 任务执行完毕后方可执行，t2 任务也是在 t0 任务执行完毕后方可执行，t3 任务在 t1、t2 任务全部执行完毕后方可执行。请（1）参考题目 2.1 中的图，绘图表示任务之间的前驱后继关系（2）设计信号量机制，用 PV 操作控制任务间的前驱后继关系。信号量用得越少越好。并发度越高越好。



```
semaphore s1, s2;
s1.value = 0;
s2.value = 0;

t0()
{
    task t0;
    V(s1); // 给 t1 一个通过信号
    V(s1); // 给 t2 一个通过信号
}

t1()
{
    P(s1); // 等待 t0 完成
    task t1;
    V(s2); // 通知 t3: t1 已完成
}

t2()
{
    P(s1); // 等待 t0 完成
    task t2;
    V(s2); // 通知 t3: t2 已完成
}

t3()
{
    P(s2); // 等待 t1 或 t2 中的一个完成
    P(s2); // 等待另一个也完成
    task t3;
}
```


3、生产者、消费者问题

3.1 生产者进程互斥 $P(\text{empty})$ 和 $P(\text{mutex})$ 操作并且消费者进程互斥 $P(\text{full})$ 和 $P(\text{mutex})$ 操作之后，系统有可能出现死锁：生产者进程和消费者进程彼此等待，谁也无法前进的僵持情况。试分析系统可能出现死锁的所有情况。

死锁情况一：缓冲区为空时，消费者先进入

$empty = n$

$full = 0$

$mutex = 1$

步骤	执行者	操作	结果
1	消费者	$P(mutex)$	$mutex$ 由 1 变 0，消费者进入临界区
2	消费者	$P(full)$	$full$ 为 0，没有产品，消费者阻塞
3	生产者	$P(mutex)$	$mutex$ 已被消费者占用，生产者阻塞

- 消费者等待 $full > 0$ ，即等待生产者生产产品；
- 生产者等待 $mutex$ ，即等待消费者释放缓冲区互斥锁；
- 但消费者已经阻塞在 $P(full)$ ，不会执行 $V(mutex)$ ；
- 所以双方互相等待，发生死锁。

死锁情况二：缓冲区满时，生产者先进入

$empty = 0$

$full = n$

$mutex = 1$

步骤	执行者	操作	结果
1	生产者	$P(mutex)$	$mutex$ 由 1 变 0，生产者进入临界区
2	生产者	$P(empty)$	$empty$ 为 0，没有空位，生产者阻塞
3	消费者	$P(mutex)$	$mutex$ 已被生产者占用，消费者阻塞

- 生产者等待 $empty > 0$ ，即等待消费者消费一个产品；
- 消费者等待 $mutex$ ，即等待生产者释放缓冲区互斥锁；
- 但生产者已经阻塞在 $P(empty)$ ，不会执行 $V(mutex)$ ；
- 所以消费者无法消费，也就不会执行 $V(empty)$ ；
- 生产者永远等不到空缓冲区，发生死锁。

3.2 深刻理解实现。此题有很多变形。

3.3*** 有一个仓库存放两种零件 A 和 B，最大容量各为 m 个。有一车间不断地取 A 和 B 进行装配，每次各取一个。为了避免零件锈蚀，遵循先入库者先出库的原则。有两组供应商

分别不断地供应 A 和 B（每次一个），为保证齐套和合理库存，当某种零件的数量比另一种数量少超过 n ($n < m$) 个时，暂停对数量大的零件的进货，集中补充数量少的零件。

```
item bufferA[m], bufferB[m];
int inA = 0, outA = 0;
int inB = 0, outB = 0;

semaphore emptyA = m, emptyB = m;
semaphore fullA = 0, fullB = 0;
semaphore sa = n, sb = n;
semaphore mutex = 1;

cobegin
process producerA(){
    while (true)
    {
        produce A;
        P(emptyA);
        P(sa);
        P(mutex);
        bufferA[inA] = A;
        inA = (inA + 1) % m;
        V(mutex);
        V(sb);
        V(fullA);
    }
}
process producerB()
{
    while (true)
    {
        produce B;
        P(emptyB);
        P(sb);
        P(mutex);
        bufferB[inB] = B;
        inB = (inB + 1) % m;
        V(mutex);
        V(sa);
        V(fullB);
    }
}
process workshop()
```

```

{
    while (true)
    {
        P(fullA);
        P(fullB);
        P(mutex);
        take bufferA[outA];
        take bufferB[outB];
        outA = (outA + 1) % m;
        outB = (outB + 1) % m;
        V(mutex);
        V(emptyA);
        V(emptyB);
        assemble product;
    }
}
coend

```

4、读者、写者问题

4.1 下面是一个多读者多写者对同一块数据区并发访问的伪代码，阅读代码后回答问题：

```

int readcount = 0;          /* 记录正在读的进程数目*/
semaphore wmutex;
wmutex.value = 1;          /* 写互斥信号量*/

```

```

void writer()
{
    while(true) {
        p(wmutex);
        WRITEUNIT(); /* 写操作*/
        v(wmutex);
    }
}

```

```

void reader()
{
    while(true) {

        readcount++;
        if (readcount == 1)
            p(wmutex);

        READUNIT();

        readcount--;
        if (readcount == 0)
            v(wmutex);

    }
}

```

(1) 请标出上述代码中的临界区。**代码中黄色高亮区域**

(2) 上述代码是否对所有的临界资源都实施了有效的保护，如果没有，请尝试添加必要的信号量，并在代码的适当位置添加对该信号量的 P，V 操作。

没有保护好共享变量 readcount

需要增加一个互斥信号量 rmutex，用它保护 readcount。

```

int readcount = 0;

semaphore wmutex;
semaphore rmutex;

wmutex.value = 1; /* 控制读者/写者对共享数据区的互斥 */
rmutex.value = 1; /* 控制对 readcount 的互斥访问 */

void writer()
{
    while (true)
    {
        p(wmutex);

```

```

        WRITEUNIT(); /* 写操作 */
        v(wmutex);
    }
}

void reader()
{
    while (true)
    {
        p(rmutex);
        readcount++;
        if (readcount == 1)
            p(wmutex);
        v(rmutex);
        READUNIT(); /* 读操作 */
        p(rmutex);
        readcount--;
        if (readcount == 0)
            v(wmutex);
        v(rmutex);
    }
}

```

(3) 修复的代码是否会出现写者饿死的情况，请说明。

会出现。因为该算法是读者优先算法。只要当前有读者在读，新的读者就可以继续进入。如果在最后一个读者退出之前总是有新的读者到来，那么 `readcount` 一直不会变为 0，`wmutex` 一直不会被释放，写者就会一直阻塞在 `p(wmutex)` 上，最终发生写者饿死。

(4) 什么是死锁。写出系统产生死锁的 4 个必要条件（必做）。写者饿死的情况是死锁嘛？对照产生死锁的四个必要条件，解释你的判断。

死锁是指一组进程中的每个进程都在等待只能由该组中其他进程释放的资源，导致这些进程都无法继续执行的状态。

产生死锁的四个必要条件是：

- (1) 互斥条件：某些资源一次只能被一个进程使用。
- (2) 请求并保持条件：进程已经占有某些资源，同时又请求新的资源，并且等待时不释放已经占有的资源。

(3) 不可剥夺条件：进程已经获得的资源在使用完之前不能被强制剥夺，只能由进程主动释放。

(4) 循环等待条件：存在一个进程等待环，每个进程都在等待下一个进程占有的资源。

写者饿死不是死锁。因为在写者饿死时，写者虽然一直等待 wmutex，但读者进程仍然可以继续执行，系统没有全部阻塞。同时，写者只是等待 wmutex，并没有占有某个资源后再请求其他资源，所以不满足请求并保持条件；读者和写者之间也没有形成循环等待。因此写者饿死属于饥饿现象，不属于死锁。

5、嗜眠理发师

(1) 快餐店有 n 个座位。顾客光临时，若座位全满，转身离开。请问，下面的解正确吗？如果有错，错在哪里。注释：代码中 seat 是空座位的数量。



customer 进程

```
semaphore mutex= 1;
int seat=n;

p(mutex);
if(seat== 0)
    goto out;
else
    seat--;
v(mutex);

eating;

p(mutex);
seat++;
v(mutex);

out:
exit;
```

不正确。错误在于：当 `seat == 0` 时，说明座位已满，顾客应该离开。但是该顾客在执行 `P(mutex)` 获得互斥信号量后，直接 `goto out` 离开，没有执行 `V(mutex)` 释放互斥信号量。

这样会导致 `mutex` 一直被占有。之后店内已经在吃饭的顾客吃完后需要执行 `P(mutex)` 来

修改 seat，但由于 mutex 没有被释放，会被阻塞，无法执行 seat++；店外新来的顾客也无法执行 P(mutex) 进入判断。因此会造成店里的顾客出不来，外面的顾客进不去，系统陷入阻塞。

正确做法是在 goto out 之前加上 V(mutex)

(2) 复印室里有一个操作员为顾客复印资料，有 5 把椅子供顾客休息等待复印。如果没有顾客，则操作员休息。当顾客来到复印室时，如果有空椅子则坐下来，并唤醒复印操作员；如果没有空椅子则必须离开复印室。

设 mutex 为互斥信号量，用于保护等待顾客数 waiting；customers 表示等待复印的顾客数；operator 表示操作员是否可以为顾客服务。初值为 mutex=1，customers=0，operator=0，waiting=0。

当顾客到来时，先互斥检查 waiting 是否小于 5。若 waiting<5，则顾客坐下等待，waiting 加 1，并执行 V(customers) 唤醒操作员；然后执行 P(operator) 等待操作员叫号。如果 waiting=5，说明没有空椅子，顾客释放 mutex 后离开。

操作员进程不断执行 P(customers)。如果没有顾客，则阻塞休息；如果有顾客，则醒来，互斥地使 waiting 减 1，表示叫一个顾客来复印，并执行 V(operator) 通知该顾客，然后为其复印资料。

```
semaphore mutex = 1;
semaphore customers = 0;
semaphore operator = 0;

int waiting = 0;
int chairs = 5;
void operator_process()
{
    while (true) {
        P(customers);    // 没有顾客时，操作员睡眠
        P(mutex);
        waiting--;
        V(operator);
        V(mutex);
        copy();
    }
}
void customer()
{
    P(mutex);
```



```
    if (waiting < chairs) {
        waiting++;        // 顾客坐下等待
        V(customers);     // 通知操作员有顾客来了
        V(mutex);
        P(operator);      // 等待操作员叫号
        get_copy();
    }
    else {                 // 没有空椅子，释放锁后离开
        V(mutex);
        leave();
    }
}

void main()
{
    cobegin
        operator_process();
        customer();
        customer();
        ...
    coend
}
```

(3) 自助复印室有 5 把椅子供顾客休息等待。顾客 FIFO 使用复印机。复印机有人使用，坐在椅子上等待。没有空椅子，转身离开。

为了保证顾客按照先到先服务（FIFO）的顺序使用复印机，需要设置一个等待队列 `waitQueue`。顾客到达时，如果复印机空闲，则直接使用复印机；如果复印机正在被使用，并且还有空椅子，则该顾客坐下等待，同时将自己的私有信号量 `self` 加入等待队列队尾。由于等待队列按照先进先出的顺序组织，因此先到达的顾客会排在队列前面。

每个等待顾客都有一个私有信号量 `self`，初值为 0。顾客入队后执行 `P(self)` 阻塞，表示等待轮到自己使用复印机。当正在使用复印机的顾客复印完毕后，检查等待队列是否为空。如果队列非空，则取出队首顾客的私有信号量 `next`，并执行 `V(next)`，唤醒队首顾客。这样，只有排在最前面的顾客会被唤醒并使用复印机，从而保证了严格的 FIFO 顺序。

变量 `freeSeat` 表示空椅子的数量。当顾客坐下等待时，`freeSeat` 减 1；当顾客被唤醒并离开椅子去使用复印机时，`freeSeat` 加 1。变量 `copierBusy` 表示复印机是否正在被使用。当没有顾客等待时，复印机使用完毕后将 `copierBusy` 置为 `false`；如果仍有顾客等待，则直接唤醒队首顾客，复印机使用权交给该顾客。

`mutex` 用于互斥访问共享变量 `freeSeat`、`copierBusy` 以及等待队列 `waitQueue`，防止多个顾客同时修改这些共享数据而产生错误。因此，该方案既保证了复印机一次只能被一个顾客使用，又保证了等待顾客按先到先服务的顺序使用复印机；当没有空椅子时，新来的顾客释放 `mutex` 后直接离开，不会影响系统正常运行。

```
semaphore mutex = 1;
int freeSeat = 5;
bool copierBusy = false;
queue<semaphore> waitQueue;

void customer()
{
    semaphore self = 0; // 当前顾客自己的私有信号量
    P(mutex);
    if (copierBusy == false)
    {
        // 复印机空闲，顾客直接使用复印机
        copierBusy = true;
        V(mutex);
```

```

    copy(); // 使用复印机
    P(mutex);
    if (!waitQueue.empty())
    {
        semaphore next = waitQueue.front();
        waitQueue.pop();
        // 唤醒队首顾客，复印机使用权交给他
        V(next);
    }
    else
    {
        copierBusy = false;
    }
    V(mutex);
}
else
{
    // 复印机正在被使用
    if (freeSeat > 0)
    {
        // 有空椅子，顾客坐下等待
        freeSeat--;
        waitQueue.push(self);
        V(mutex);
        P(self); // 等待被前一个顾客唤醒
        // 被唤醒后，说明轮到自己使用复印机
        P(mutex);
        freeSeat++; // 离开等待椅
        V(mutex);
        copy(); // 使用复印机
        P(mutex);
        if (!waitQueue.empty())
        {
            semaphore next = waitQueue.front();
            waitQueue.pop();
            // 唤醒下一个等待顾客
            V(next);
        }
        else
        {
            copierBusy = false;
        }
        V(mutex);
    }
}
}

```

```
else
{
    // 没有空椅子，顾客离开
    V(mutex);
    leave();
}
}
```